

EXP 6

Aim: Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE.

Theory:

1. Relevance of require Statements in Solidity Programs

In Solidity, the require statement acts as a **guard condition** within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside require is not satisfied, function execution stops immediately and all state changes made during that transaction are **reverted** to their original state. This rollback mechanism ensures that invalid transactions do not corrupt the blockchain data.

Syntax:

```
solidity
require(condition, "Error message");
```

Uses in the Voting (Ballot) contract:

- Checking whether the caller has the right to vote:

```
solidity
require(voters[msg.sender].weight > 0, "Has no right to vote");
```

- Checking whether a voter has already voted before allowing them to vote again.
- Checking whether the function caller is the chairperson before granting voting rights.

Conclusion: require statements enforce security, correctness, and reliability in smart contracts. They also let developers attach error messages, which makes debugging and interacting with the contract easier.

2. Keywords: mapping, storage, and memory

a) mapping

A mapping is a special data structure in Solidity that links keys to values, similar to a hash table.

Syntax: mapping(keyType => valueType)

Example:

```
mapping(address => Voter) public voters;
```

Here, each address (Ethereum account) is mapped to a Voter structure. This is useful in the Ballot contract, where each voter must be associated with their data (whether they voted, which proposal they chose, and so on).

Unlike arrays, mappings have no length property and cannot be iterated over directly. This makes them gas-efficient for lookups but limited for enumeration.

b) storage

storage is the **permanent** memory of the contract, kept on the Ethereum blockchain. Variables declared at the contract level are stored in storage by default.

- Data persists across transactions and remains available unless explicitly modified.
- Writing to storage consumes gas, so it is **expensive**.
- Example: a voter's information saved in the voters mapping remains available throughout the contract's lifecycle.

c) memory

memory is **temporary** storage that exists only for the duration of a function call. When execution ends, the data is discarded.

- Used for temporary variables, function arguments, and computations that don't need to be stored permanently.
- It is **cheaper** than storage in terms of gas.
- Example: the proposalNames array passed to the constructor is held in memory.

Comparison

Feature	storage	memory
Lifetime	Permanent (on blockchain)	Temporary (during function call)
Gas cost	High	Low
Default for	Contract-level variables	Function arguments and local variables of reference types (must be declared explicitly)
Example	mapping(address => Voter) voters	bytes32[] memory proposalNames

Conclusion: A smart contract developer must balance storage and memory to keep the contract efficient and cost-effective.

3. Why bytes32 Instead of string?

In the original Ballot contract, bytes32 is used for proposal names instead of string. The reason is **efficiency and gas optimization**.

bytes32

- A fixed-size type that always holds exactly 32 bytes.
- Storage is simple, comparisons are faster, and gas costs are lower.
- Limitation: names are restricted to 32 characters, which is less flexible and less user-friendly.

string

- A dynamically sized type that can hold text of any length.
- Easier for developers and users, since names can be written normally.
- Requires more complex handling inside the Ethereum Virtual Machine (EVM), which increases gas usage and can slow comparisons and manipulations.

Feature	bytes32	string
Size	Fixed (32 bytes)	Dynamic
Gas cost	Lower	Higher
Readability	Low (hex input)	High
Best for	Performance and gas efficiency	Readability and ease of use

To make the system more user-friendly, tools like the **Web3 Type Converter** can convert between bytes32 and string for deployment and testing.

Code:

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

/**
 * @title Ballot
 * @dev Implements voting process along with vote delegation
 */
contract Ballot {

    // Represents a single voter.
    struct Voter {
        uint weight; // weight is accumulated by delegation
        bool voted; // if true, that person already voted
        address delegate; // person delegated to
        uint vote; // index of the voted proposal
    }
```

```

// Represents a single proposal.
struct Proposal {
    bytes32 name; // short name (up to 32 bytes)
    uint voteCount; // number of accumulated votes
}

address public chairperson;

// Stores a 'Voter' struct for each possible address.
mapping(address => Voter) public voters;

// A dynamically-sized array of 'Proposal' structs.
Proposal[] public proposals;

// Events make transactions easy to follow in the Remix console/logs.
event RightGiven(address indexed voter);
event Delegated(address indexed from, address indexed to);
event Voted(address indexed voter, uint indexed proposal, uint weight);

/**
 * @dev Create a new ballot to choose one of 'proposalNames'.
 * @param proposalNames names of proposals
 */
constructor(bytes32[] memory proposalNames) {
    require(proposalNames.length > 0, "Provide at least one proposal.");

    chairperson = msg.sender;
    voters[chairperson].weight = 1;

    for (uint i = 0; i < proposalNames.length; i++) {
        proposals.push(Proposal({
            name: proposalNames[i],
            voteCount: 0
        }));
    }
}

/**
 * @dev Give 'voter' the right to vote on this ballot. May only be called by 'chairperson'.
 * @param voter address of voter
 */
function giveRightToVote(address voter) external {
    require(
        msg.sender == chairperson,

```

```

    "Only chairperson can give right to vote."
);
require(!voters[voter].voted, "The voter already voted.");
require(voters[voter].weight == 0, "Voter already has the right to vote.");

voters[voter].weight = 1;
emit RightGiven(voter);
}

/**
 * @dev Delegate your vote to the voter 'to'.
 * @param to address to which vote is delegated
 */
function delegate(address to) external {
    Voter storage sender = voters[msg.sender];
    require(sender.weight != 0, "You have no right to vote.");
    require(!sender.voted, "You already voted.");
    require(to != msg.sender, "Self-delegation is disallowed.");

    // Forward the delegation as long as 'to' also delegated.
    // Such loops can be dangerous (they may run out of gas),
    // but here the transaction would simply revert.
    while (voters[to].delegate != address(0)) {
        to = voters[to].delegate;
        // A loop in the delegation is not allowed.
        require(to != msg.sender, "Found loop in delegation.");
    }

    Voter storage delegate_ = voters[to];

    // Voters cannot delegate to accounts that cannot vote.
    require(delegate_.weight >= 1, "Delegate has no right to vote.");

    // 'sender' is a reference, so this modifies 'voters[msg.sender]'.
    sender.voted = true;
    sender.delegate = to;

    if (delegate_.voted) {
        // Delegate already voted: add directly to the proposal's count.
        proposals[delegate_.vote].voteCount += sender.weight;
    } else {
        // Delegate has not voted yet: add to their weight.
        delegate_.weight += sender.weight;
    }
}

```

```

    emit Delegated(msg.sender, to);
}

/**
 * @dev Give your vote (including votes delegated to you) to proposal 'proposals[proposal].name'.
 * @param proposal index of proposal in the proposals array
 */
function vote(uint proposal) external {
    Voter storage sender = voters[msg.sender];
    require(sender.weight != 0, "Has no right to vote.");
    require(!sender.voted, "Already voted.");
    require(proposal < proposals.length, "Invalid proposal index.");

    sender.voted = true;
    sender.vote = proposal;

    proposals[proposal].voteCount += sender.weight;

    emit Voted(msg.sender, proposal, sender.weight);
}

/**
 * @dev Computes the winning proposal taking all previous votes into account.
 * In case of a tie, the proposal with the lowest index wins.
 * @return winningProposal_ index of winning proposal in the proposals array
 */
function winningProposal() public view returns (uint winningProposal_) {
    uint winningVoteCount = 0;
    for (uint p = 0; p < proposals.length; p++) {
        if (proposals[p].voteCount > winningVoteCount) {
            winningVoteCount = proposals[p].voteCount;
            winningProposal_ = p;
        }
    }
}

/**
 * @dev Calls winningProposal() to get the index of the winner and returns its name.
 * @return winnerName_ the name of the winner
 */
function winnerName() external view returns (bytes32 winnerName_) {
    winnerName_ = proposals[winningProposal()].name;
}

```

```

/**
 * @dev Helper: number of proposals (handy for testing in Remix).
 */
function proposalCount() external view returns (uint) {
    return proposals.length;
}
}

```

Output:

Contract Deployed:

The screenshot shows the Remix IDE interface. On the left, the 'Deploy' tab is active, displaying a list of transactions. The 'vote' transaction is selected, showing its parameters and a 'Transact' button. The main editor displays the Solidity code for the 'VotingBallot' contract. The bottom panel shows the transaction history with a successful deployment of the 'Ballot.sol' contract.

Owner of Contract:

The screenshot shows the Remix IDE interface. On the left, the 'Call' button is active, showing the 'owner' property of the contract. The main editor displays the Solidity code for the 'VotingBallot' contract. The bottom panel shows the transaction history with a successful call to the 'owner' function.

Added Alice as Candidate:

The screenshot shows the Remix IDE interface. On the left, the 'Functions' panel lists 'addCandidate (string)', 'vote (uint256)', and 'candidates (uint256)'. The 'addCandidate' function is selected, and its parameters are set to 'Alice'. The 'Transact' button is clicked. The main editor shows the Solidity code for the 'VotingBallot' contract, including the 'addCandidate' function. The 'Transactions history' panel on the right shows the transaction details, including the transaction hash, value, data, logs, and hash.

Added Bob Candidate:

The screenshot shows the Remix IDE interface. On the left, the 'Functions' panel lists 'addCandidate (string)', 'vote (uint256)', and 'candidates (uint256)'. The 'addCandidate' function is selected, and its parameters are set to 'Bob'. The 'Transact' button is clicked. The main editor shows the Solidity code for the 'VotingBallot' contract, including the 'addCandidate' function. The 'Transactions history' panel on the right shows the transaction details, including the transaction hash, value, data, logs, and hash.

Added Charlie Candidate:

The screenshot shows the Remix IDE interface. On the left, the 'Functions' panel lists 'addCandidate (string)', 'vote (uint256)', and 'candidates (uint256)'. The 'addCandidate' function is selected, and its parameters are set to 'Charlie'. The 'Transact' button is clicked. The main editor shows the Solidity code for the 'VotingBallot' contract, including the 'addCandidate' function. The 'Transactions history' panel on the right shows the transaction details, including the transaction hash, value, data, logs, and hash.

Calling getCandidateCount:

The screenshot shows the Remix IDE interface. On the left, the 'Functions' panel lists 'getCandidateCount', 'getWinner', 'hasVoted', and 'owner'. The 'getCandidateCount' function is selected, and its parameters are set to '0'. The 'Call' button is clicked. The main editor shows the Solidity code for the 'VotingBallot' contract, including the 'getCandidateCount' function. The 'Transactions history' panel on the right shows the transaction details, including the transaction hash, value, data, logs, and hash.

Calling same on Alice:

The screenshot shows the Remix IDE interface. On the left, the 'Deployed contracts' panel lists 'candidates' (uint256), 'getCandidate' (uint256), 'getCandidateCount', 'getWinner', and 'hasVoted' (address). The 'Transactions history' panel shows a list of transactions. The main editor displays the Solidity code for the 'VotingBallot' contract, which includes a 'Candidate' struct, a 'candidates' array, and a 'hasVoted' mapping. The transaction history shows a successful transaction from 0x58380d6a701c568545dcfc803fc8875f56bedd4 to VotingBallot.addCandidate(string) @0x91...39138 with a value of 0 wei and data 0x462...00000. Below this, two 'CALL' transactions are shown: one from 0x58380d6a701c568545dcfc803fc8875f56bedd4 to VotingBallot.getCandidateCount() and another to VotingBallot.getCandidate(uint256).

Alice has 0 votes

Voting for Alice from acc 2:

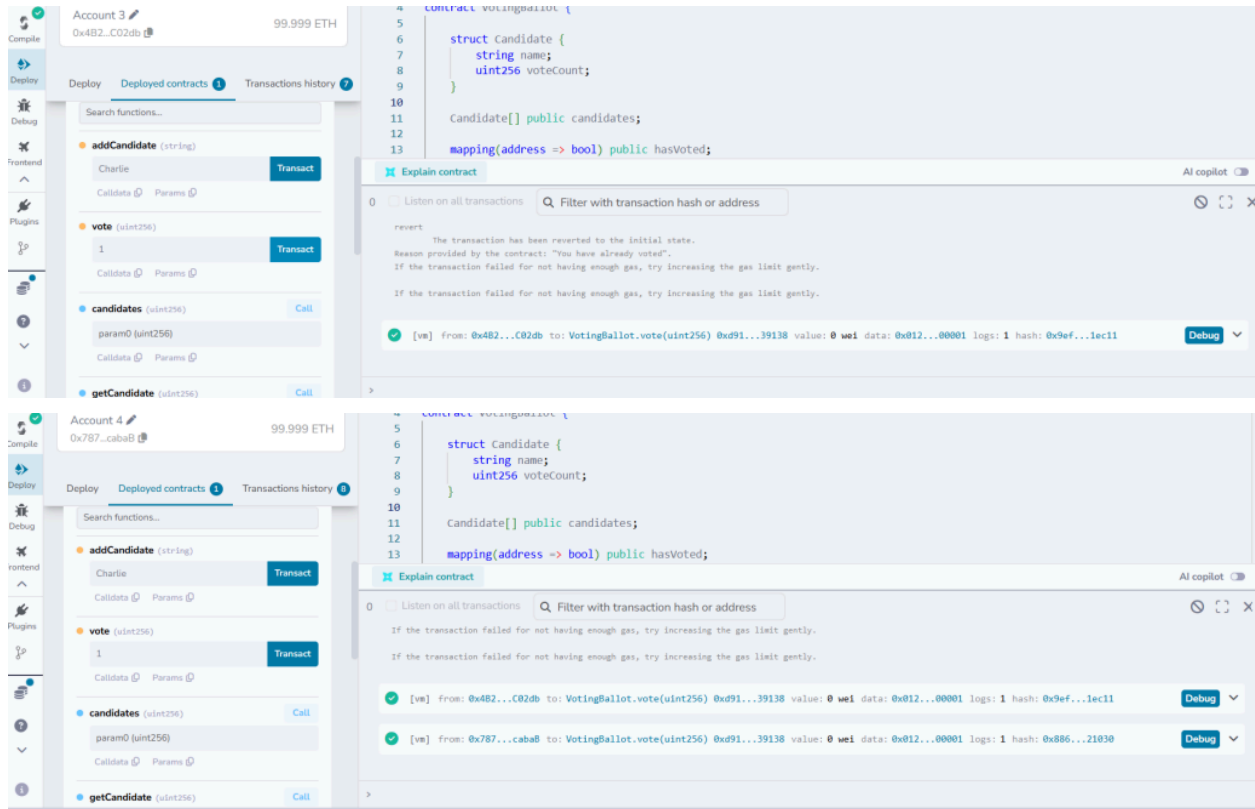
The screenshot shows the Remix IDE interface. On the left, the 'Deployed contracts' panel lists 'vote' (uint256), 'candidates' (uint256), and 'getCandidate' (uint256). The 'Transactions history' panel shows a list of transactions. The main editor displays the Solidity code for the 'VotingBallot' contract. The transaction history shows a successful transaction from 0x58380d6a701c568545dcfc803fc8875f56bedd4 to VotingBallot.getCandidateCount() and another to VotingBallot.getCandidate(uint256). Below these, a transaction from 0xA08...35cb2 to VotingBallot.vote(uint256) @0x91...39138 is shown with a value of 0 wei and data 0x012...00000. The transaction is marked as successful.

Preventing double voting:

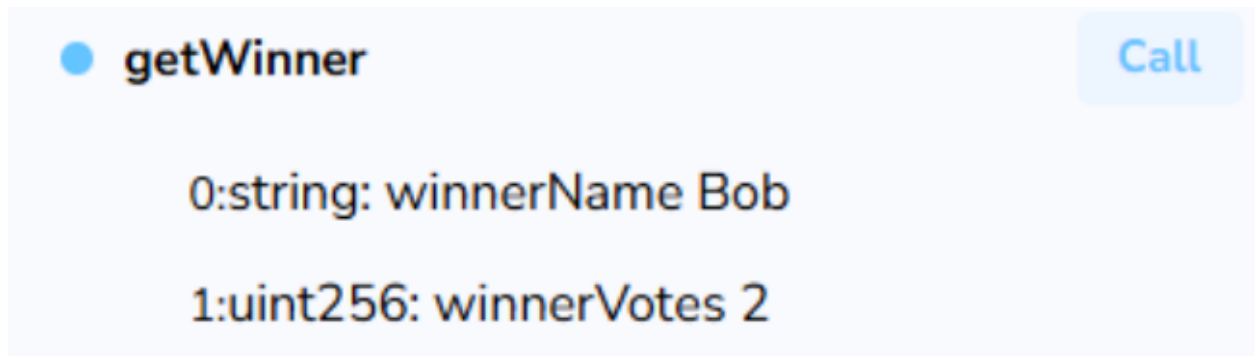
The screenshot shows the Remix IDE interface. On the left, the 'Deployed contracts' panel lists 'vote' (uint256), 'candidates' (uint256), and 'getCandidate' (uint256). The 'Transactions history' panel shows a list of transactions. The main editor displays the Solidity code for the 'VotingBallot' contract. The transaction history shows a transaction from 0xA08...35cb2 to VotingBallot.vote(uint256) @0x91...39138 with a value of 0 wei and data 0x012...00000. This transaction is marked as failed with a red 'X' icon. Below the failed transaction, a message states: 'transaction to VotingBallot.vote errored: Error occurred: Error occurred: revert. The transaction has been reverted to the initial state. Reason provided by the contract: "You have already voted". If the transaction failed for not having enough gas, try increasing the gas limit gently.'

Double voting prevented successfully

Using 3rd and 4th account to vote for Bob(1):



Finding winner:



Conclusion:

In this experiment, a Voting/Ballot smart contract was deployed using Solidity on the Remix IDE. The concepts of require statements, mapping, and data location specifiers like storage and memory were explored to understand their role in ensuring security, efficiency, and correctness in smart contracts. The difference between using bytes32 and string for proposal names was also studied, highlighting the trade-off between gas efficiency and readability. Overall, the experiment provided practical insights into the design and

deployment of voting contracts on the blockchain.